

School of Computer Science and Artificial Intelligence**Lab Assignment # 13.2**

Program : B. Tech (CSE)
Specialization : --
Course Title : AI Assisted coding
Course Code :
Semester II
Academic Session : 2025-2026
Name of Student : A.VamshiKrishna
Enrollment No. : 2403A51L47
Batch No. : 52
Date :10-03-2026

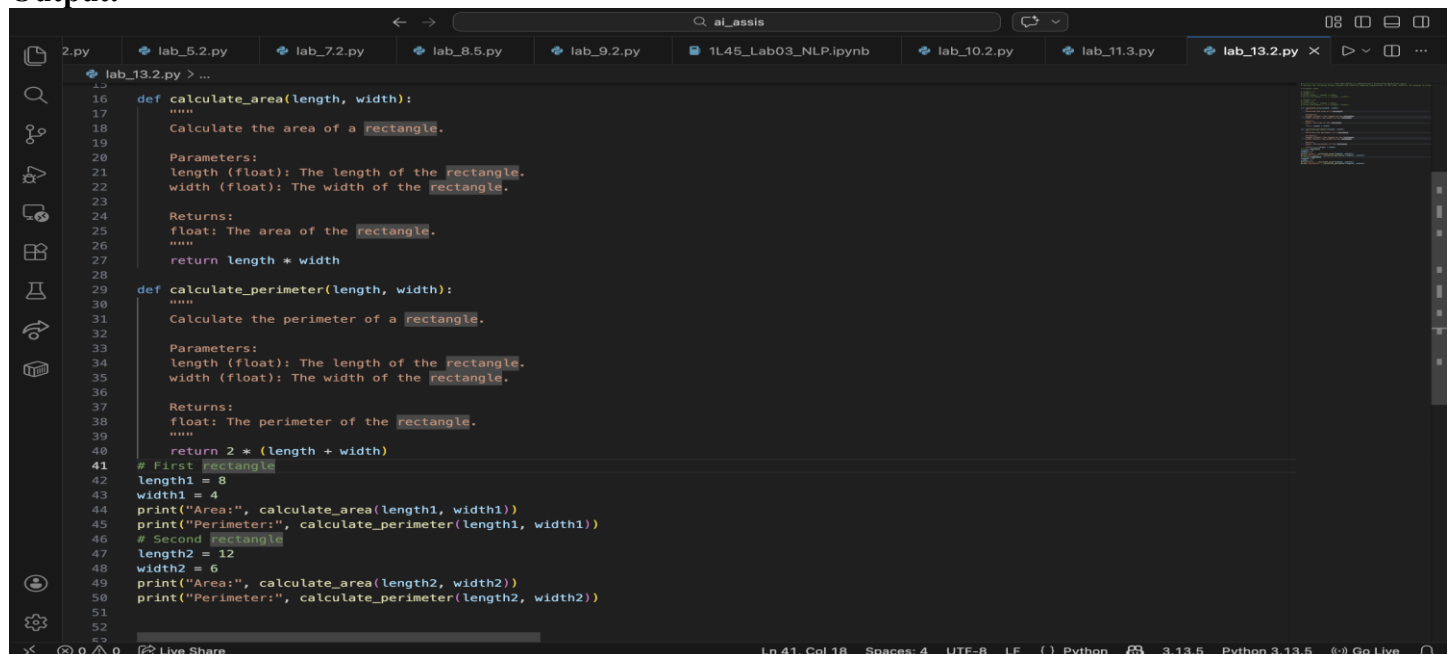
Task 1 (Refactoring – Eliminating Repetitive Logic):

Prompt:Analyze the following Python program and identify repeated computations in the code. Refactor the program by eliminating duplicated logic and encapsulating the repeated calculations into a reusable function. Ensure that the output of the program remains exactly the same after refactoring. Also add clear and proper docstrings to all defined functions explaining their purpose, parameters, and return values.

Original Code:

```
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

```
length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

Output:

```
16 def calculate_area(length, width):
17     """
18     Calculate the area of a rectangle.
19
20     Parameters:
21     length (float): The length of the rectangle.
22     width (float): The width of the rectangle.
23
24     Returns:
25     float: The area of the rectangle.
26     """
27     return length * width
28
29 def calculate_perimeter(length, width):
30     """
31     Calculate the perimeter of a rectangle.
32
33     Parameters:
34     length (float): The length of the rectangle.
35     width (float): The width of the rectangle.
36
37     Returns:
38     float: The perimeter of the rectangle.
39     """
40     return 2 * (length + width)
41
42 # First rectangle
43 length1 = 8
44 width1 = 4
45 print("Area:", calculate_area(length1, width1))
46 print("Perimeter:", calculate_perimeter(length1, width1))
47
48 # Second rectangle
49 length2 = 12
50 width2 = 6
51 print("Area:", calculate_area(length2, width2))
52 print("Perimeter:", calculate_perimeter(length2, width2))
53
```


Task 3 (Refactoring – Improving Loop and Conditional Performance):

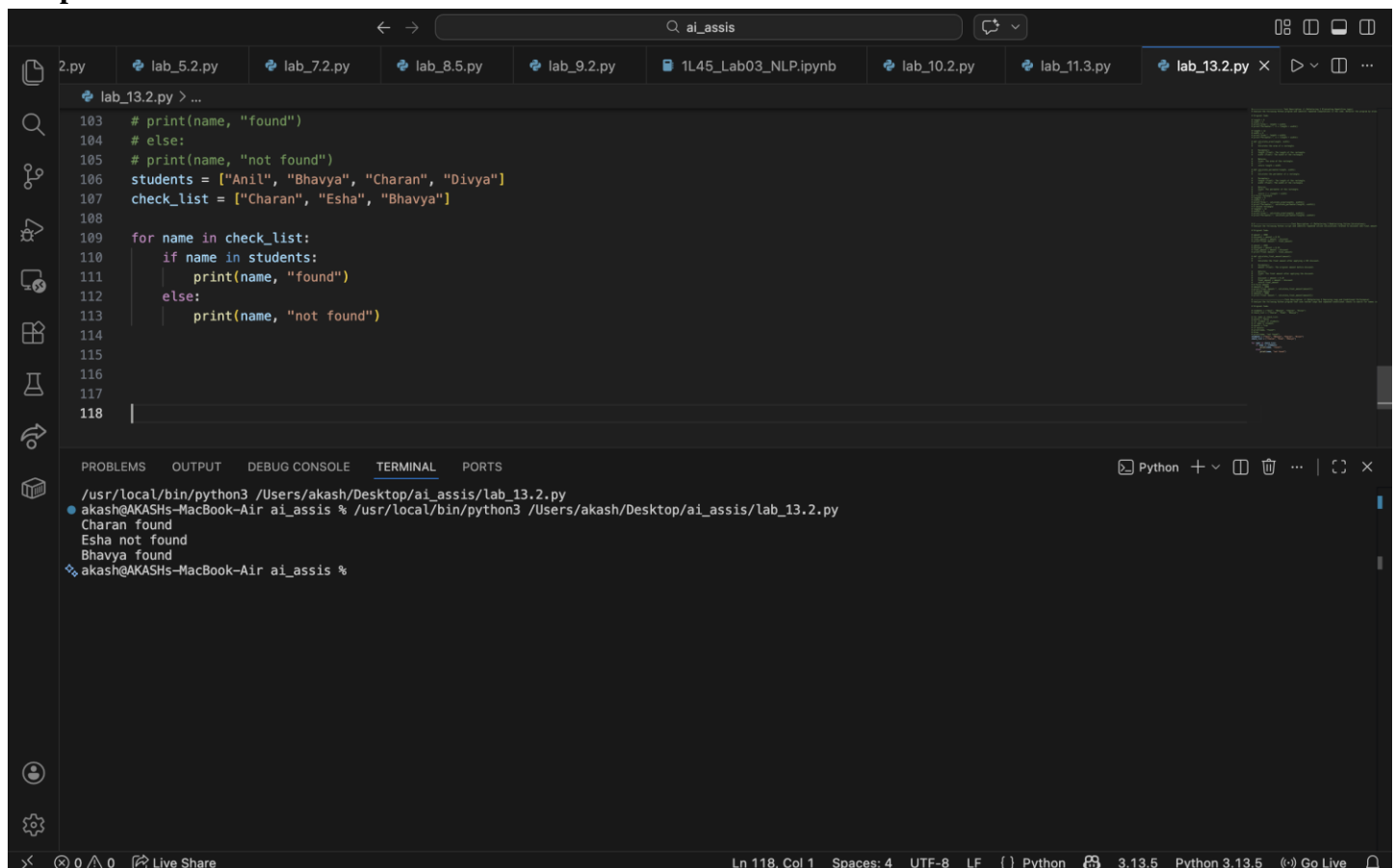
Prompt: Analyze the following Python program that uses nested loops and repeated conditional checks to search for names in a list. Identify performance issues in the code and suggest improvements using more efficient data structures or simplified logic. Refactor the program to replace inefficient constructs while preserving the original behavior and output. Additionally, compare the execution efficiency of the original and optimized versions of the code and explain the performance improvement. Ensure the refactored code remains readable and well-structured.

Original Code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
```

```
for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True
    if exists:
        print(name, "found")
    else:
        print(name, "not found")
```

Output:



The screenshot shows a Jupyter Notebook interface with a dark theme. The top bar displays the file name 'ai_assis' and various icons. Below the top bar, there are several tabs for different files, including 'lab_13.2.py'. The main editor area shows the original Python code from the prompt, with line numbers 103 to 118. The code defines two lists, 'students' and 'check_list', and uses nested loops to search for names in 'check_list' within the 'students' list. The output area at the bottom shows the execution results: 'Charan found', 'Esha not found', and 'Bhavya found'. The status bar at the very bottom indicates the current line and column (Ln 118, Col 1), the number of spaces (4), the encoding (UTF-8), the line feed (LF), the language (Python), and the version (3.13.5).

```
103 # print(name, "found")
104 # else:
105 # print(name, "not found")
106 students = ["Anil", "Bhavya", "Charan", "Divya"]
107 check_list = ["Charan", "Esha", "Bhavya"]
108
109 for name in check_list:
110     if name in students:
111         print(name, "found")
112     else:
113         print(name, "not found")
114
115
116
117
118 |
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
/usr/local/bin/python3 /Users/akash/Desktop/ai_assis/lab_13.2.py
● akash@AKASHs-MacBook-Air ai_assis % /usr/local/bin/python3 /Users/akash/Desktop/ai_assis/lab_13.2.py
Charan found
Esha not found
Bhavya found
❖ akash@AKASHs-MacBook-Air ai_assis %
```

Ln 118, Col 1 Spaces: 4 UTF-8 LF {} Python 3.13.5 Python 3.13.5 Go Live

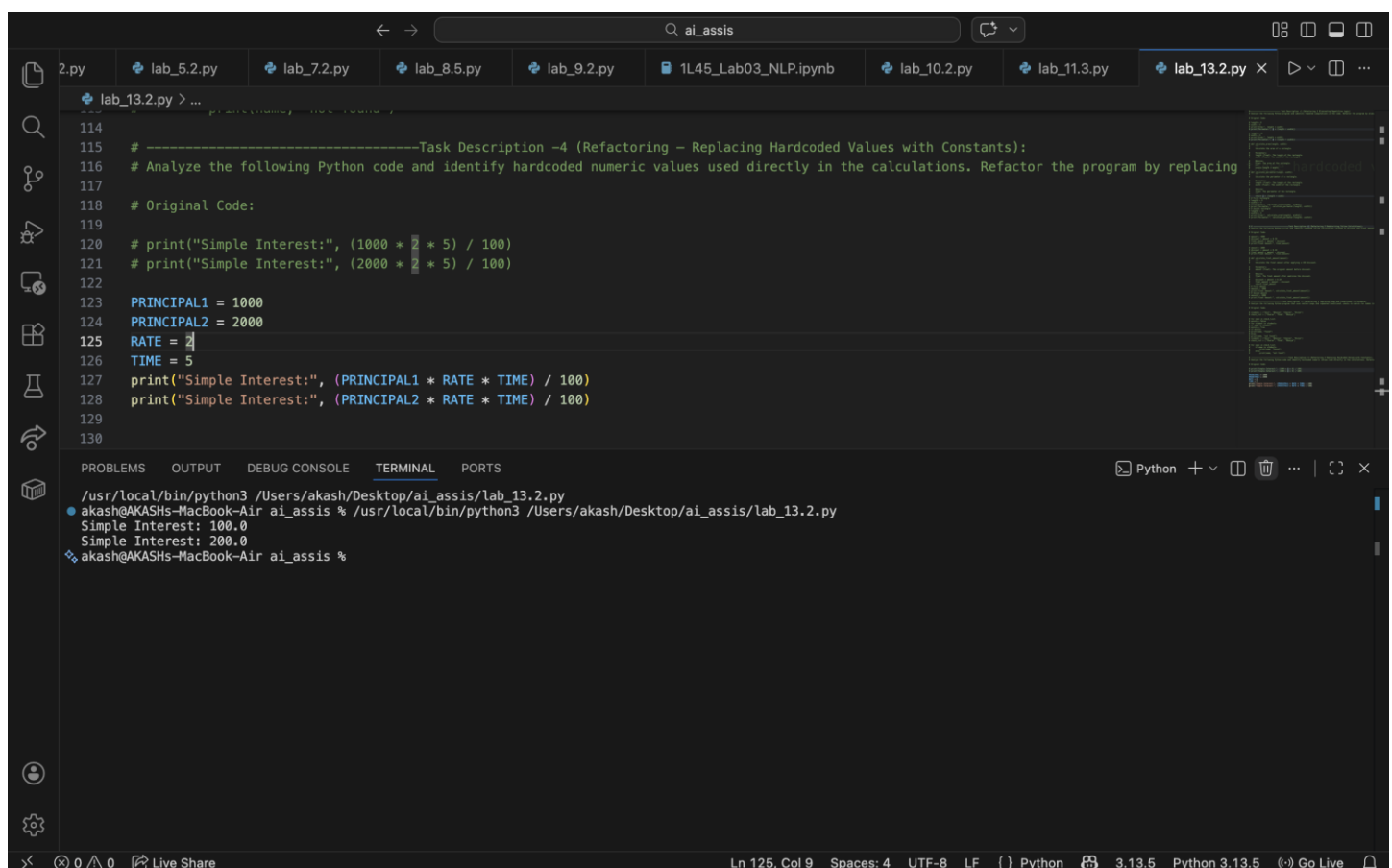
Task 4 (Refactoring – Replacing Hardcoded Values with Constants):

Prompt: Analyze the following Python code and identify hardcoded numeric values used directly in the calculations. Refactor the program by replacing these hardcoded values with descriptive named constants defined at the beginning of the program. Ensure the constants clearly represent their meaning (for example, RATE and TIME). Update the calculations to use these constants while keeping the program output unchanged. Also ensure the refactored code is readable and maintainable.

Original Code:

```
print("Simple Interest:", (1000 * 2 * 5) / 100)
print("Simple Interest:", (2000 * 2 * 5) / 100)
```

Output:



The screenshot shows a code editor with a file explorer on the left displaying several Python files. The active file, `lab_13.2.py`, contains the following code:

```
114
115 # -----Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants):
116 # Analyze the following Python code and identify hardcoded numeric values used directly in the calculations. Refactor the program by replacing
117
118 # Original Code:
119
120 # print("Simple Interest:", (1000 * 2 * 5) / 100)
121 # print("Simple Interest:", (2000 * 2 * 5) / 100)
122
123 PRINCIPAL1 = 1000
124 PRINCIPAL2 = 2000
125 RATE = 2
126 TIME = 5
127 print("Simple Interest:", (PRINCIPAL1 * RATE * TIME) / 100)
128 print("Simple Interest:", (PRINCIPAL2 * RATE * TIME) / 100)
129
130
```

The bottom panel of the editor shows the terminal output:

```
/usr/local/bin/python3 /Users/akash/Desktop/ai_assis/lab_13.2.py
● akash@AKASHs-MacBook-Air ai_assis % /usr/local/bin/python3 /Users/akash/Desktop/ai_assis/lab_13.2.py
Simple Interest: 100.0
Simple Interest: 200.0
❖ akash@AKASHs-MacBook-Air ai_assis %
```

The status bar at the bottom indicates the current line and column (Ln 125, Col 9), the number of spaces (4), the encoding (UTF-8), the line ending (LF), the language (Python), and the version (3.13.5).

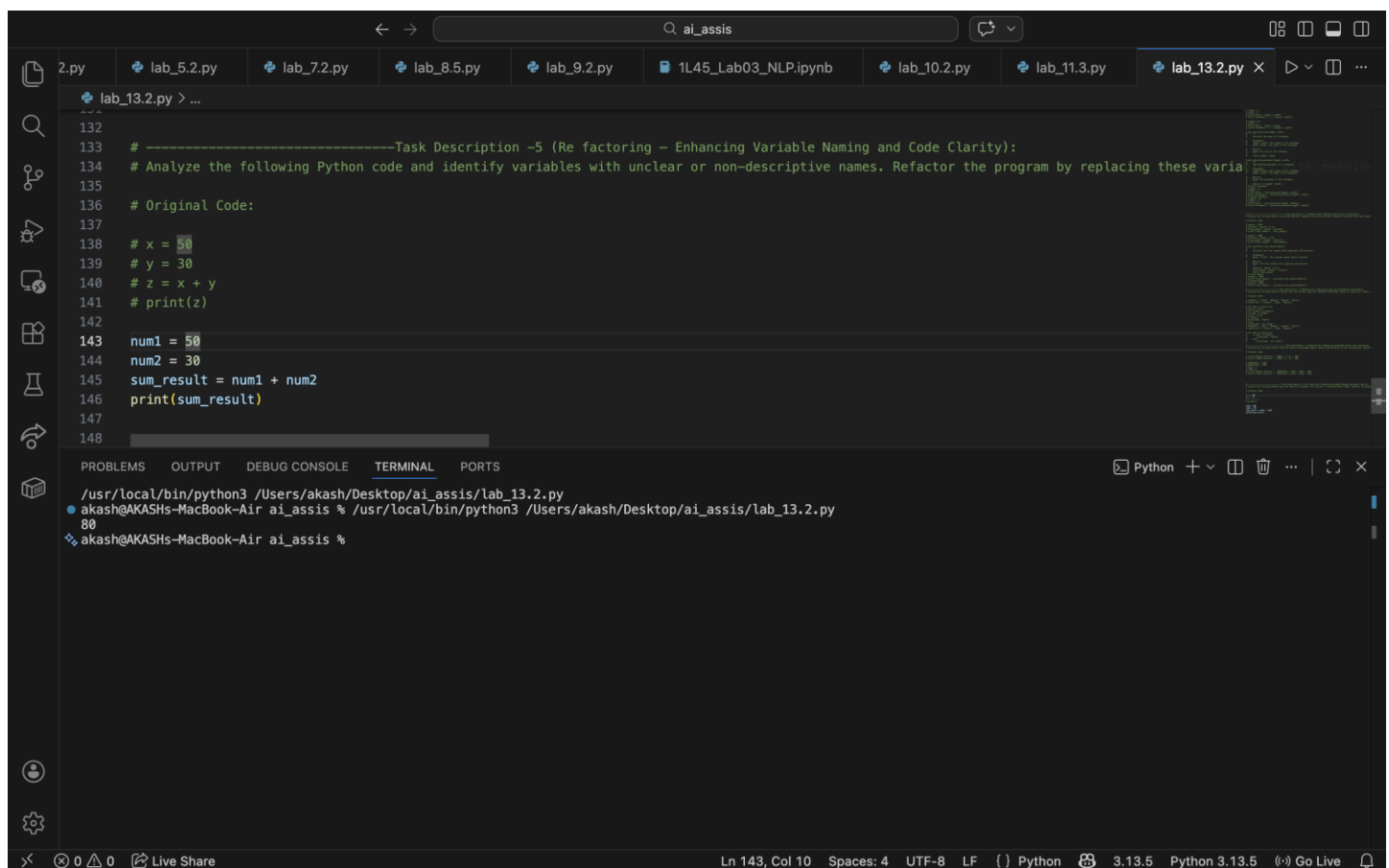
Task 5 (Re factoring – Enhancing Variable Naming and Code Clarity):

Prompt: Analyze the following Python code and identify variables with unclear or non-descriptive names. Refactor the program by replacing these variables with meaningful and descriptive identifiers that clearly represent their purpose. Add inline comments where necessary to explain the logic of the code and improve readability. Ensure that the refactored program maintains the same functionality and produces the same output as the original code.

Original Code:

```
x = 50
y = 30
z = x + y
print(z)
```

Output:



The screenshot shows a Jupyter Notebook interface with a dark theme. The top bar displays the search bar with 'ai_assis' and several open tabs: '2.py', 'lab_5.2.py', 'lab_7.2.py', 'lab_8.5.py', 'lab_9.2.py', '1L45_Lab03_NLP.ipynb', 'lab_10.2.py', 'lab_11.3.py', and 'lab_13.2.py'. The active tab is 'lab_13.2.py'. The code editor shows the following code:

```
132
133 # -----Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity):
134 # Analyze the following Python code and identify variables with unclear or non-descriptive names. Refactor the program by replacing these varia
135
136 # Original Code:
137
138 # x = 50
139 # y = 30
140 # z = x + y
141 # print(z)
142
143 num1 = 50
144 num2 = 30
145 sum_result = num1 + num2
146 print(sum_result)
147
148
```

The bottom panel shows the 'TERMINAL' output:

```
/usr/local/bin/python3 /Users/akash/Desktop/ai_assis/lab_13.2.py
● akash@AKASHs-MacBook-Air ai_assis % /usr/local/bin/python3 /Users/akash/Desktop/ai_assis/lab_13.2.py
80
● akash@AKASHs-MacBook-Air ai_assis %
```

The status bar at the bottom indicates 'Ln 143, Col 10', 'Spaces: 4', 'UTF-8', 'LF', 'Python', '3.13.5', and 'Python 3.13.5'. There is also a 'Go Live' button.